DDC @Sergey Poluyan

🚀 Planned execution

Project Bravo

- ▼ Milestone 1 @Sergey Poluyan
  - ▼ 🔴 CP1.9 Verify captured compute activity of AI agent- (roadmap)
    - **Dependencies:**
    - **Key Tasks:**
      1. Check logs when a user uses the /generate command in **Bullish Image Bot**
    - **Contributors:** @Bren
    - **References:** 💽 [ADR] Compute Resources on DDC
    - **Expected Output:**
  - ▼ 🔴 CP1.10 demonstrate how AI agent access to private data & data encryption is managed - (roadmap)
    - **Dependencies:**
    - **Key Tasks:**
      1. Show how data is encrypted & decrypted by the user key
      2. Show which AI Agent is granted access permission to that data
      3. Show NFT that defines access permissions
      4. Show/explain how key management is handled

5. Show how access can be revoked & how the agent stops working because it no longer has access

- **Contributors:** @Sergey Poluyan
- **References:**
- **Expected Output:**

▼ Milestone 3

▼ 🔴 CP3.1.1: Compute nodes must generate logs and metrics detailing the resources consumed by AI agents (e.g., GPU/CPU usage). @Sergey Poluyan @August 22, 2025

- **Dependencies**:
- **Key Tasks**:
- **Contributors**: @Sergey Poluyan
- **References:**
- **Executions**:

▼ 🔴 CP3.1.2: The cost metrics need to be clearly defined to determine how much an agent or customer should be charged for compute usage. @Sergey Poluyan @August 29, 2025

- **Dependencies**:
  -
- **Key Tasks**:
- **Contributors**: @Sergey Poluyan
- **References:**
- **Executions**:

▼ Past Checkpoints (move completed checkpoints here)

👩‍🎤 Description / Purpose (PM)

What is DDC?

Cere provides high-performance Decentralized Data Clusters (DDC) designed to shift data and AI compute to the edge.

Each DDC cluster is a complete, independent cloud. It replaces traditional infrastructure like VPC, S3, and compute services in one decentralized system with built-in on-chain verification.

Developers familiar with AWS will recognize concepts like buckets, but DDC provides the full stack: storage, CDN, compute, and usage metering. No stitching services together.

Why cubbies

We will add a two-tier memory architecture to Cubby:

1. **Hot Tier** (existing): Redis Stack for sub-millisecond working memory
2. **Cold Tier** (new): DDC-backed archive for durable long-term storage

The archive tier will support:

- Snapshot/restore of entire cubby state
- Individual object storage and retrieval
- Content-addressed immutable storage (CID-based)
- Full compatibility with distributed/decentralized topology

Why DDC?

DDC gives you one interface for storage, delivery, and compute at the edge with built-in usage verification. Instead of stitching together S3 + CloudFront + Lambda + custom metering, you get a unified stack where every operation is auditable and verified on-chain. Where traditional cloud stacks require you to trust the provider's infrastructure and logs, DDC shifts that trust that you can verify, rather than assume.

Key benefits

- **Regional & Industry Optimization:** Tailor clusters for low-latency transactional data (e.g., New Zealand) or high-bandwidth video streaming (e.g., Central

Asia).

- **Secure Data Compute**: Powered by sovereign AI frameworks (like Cef.ai), DDC enables private AI execution, ensuring that sensitive data and models remain secure at the edge.
- **Edge-Native AI**: Moves compute to the data rather than data to the cloud, reducing latency and enabling autonomous, real-time agentic workflows.

Data durability

Your data is distributed across multiple nodes using two strategies:

- Small pieces: Replication (multiple copies)
- Pieces larger than 16 KB: Erasure coding

If some nodes fail:

- Temporary outage: System routes around failures automatically
- Permanent loss: Data survives if enough copies/shares remain

See [ADR] Data redundancy strategy for technical details.

Data security

DDC provides flexible security options to meet different use cases. Data can be stored encrypted (client-side) or unencrypted, depending on your application's needs.

- **Access Control:**
  DDC provides robust access control through bucket permissions (private/public) and owner-based authentication. This protects your data from unauthorized external access.
- **Data Distribution:**
  DDC uses erasure coding to distribute your data across multiple nodes in the cluster, providing redundancy and availability. Data is split into fragments, with each node storing only a portion of your files. For a malicious actor to reconstruction a file, they require control over a significant portion of the cluster. Learn more: [ADR] Data redundancy strategy

- **Client-Side Encryption:**
  For sensitive data that requires protection at the storage level, client-side encryption is advised. By encrypting data before upload, you maintain complete control over your encryption keys, ensuring that only you can decrypt your data. Even node operators cannot access the plaintext content.

> ⚠️ Status: Client-side encryption is available manually. SDK-integrated key management with delegated access grants is in development. See [ADR] Encrypted Data Access and Key Delegation

DDC Components

- **DDC Client SDK:** Allows developers to trigger data operations and compute jobs (like AI inference) through a unified interface.
- **DDC Core:** High-performance nodes integrating decentralized storage, multi-level caching, and GPU/CPU compute power to orchestrate complex data and AI workloads.
- **DAC (Data Activity Capture):** Trust layer that captures real-time traffic and compute metrics, serving as the verified data source for blockchain-level inspection and automated payouts.
- **Tooling:** The operational gateway for developers to manage their environment, monitor usage/billing, and top up wallets for resource consumption.

▼ Old

Cere Decentralized Data Cluster offer a distributed data cloud, to be used by any self-organizing group of nodes. These nodes can execute various data operations, whether general or specific, across continents or localized to a particular region. The need for such adaptability arises from the distinct operations and economic factors in various industries and locations.

1. **Protocol & $CERE token:** Central to this decentralized approach is the Cere Protocol and the $CERE token. They are instrumental in **powering self-organizing decentralized data clusters around the world**. This is

achieved through advanced features like automated smart contract governance, real-time network monitoring, and strict SLA adherence. Such a design ensures a high standard of security and utility for these data clusters.

2. **Separation of Cloud Infra & Operations:** The tools and services offered by Cere ensure a **clean separation of cloud infrastructure & operations from the protocol**. This unique separation allows specialized data to reside on the edge, while also ensuring seamless integration and interoperability through the Cere L1 Blockchain and smart contracts.
3. **New Open Data Standard:** A significant advancement offered by the protocol is in **establishing a new open data standard**. This innovative paradigm takes various data processes such as storage, streaming, enrichment, and analysis, and decentralizes them to the edge. Such a move prepares the data ecosystem for a future dominated by autonomous data patterns.
4. **Optimization for Data Types & Regions:** The flexibility of the Cere ecosystem becomes evident when looking at cluster designs. Clusters can be **optimized for specific data types and regions**. As an illustration, one cluster might focus on large binary objects for video streaming in Central Asia, bypassing app stores or geo-restrictions. Conversely, another might cater to rapid transactional data services via a concentrated network of low-latency nodes in New Zealand.

🚚 Get started

- Quickstart: 🚚 Get started with DDC
- Current clusters:
  - Dragon 1: https://www.developer.cere.network/
- Latency/performance: https://dashboards.cere.io/d/e00352dd-8f33-4f09-af4c-11e35ded1f56/ddc-application-monitoring?orgId=1&from=now-6h&to=now&timezone=browser&var-network=mainnet&var-cluster=0059f5ada35eee46802d80750d5ca4a490640511&var-node=$__all
- Pricing: 💰 DDC Pricing & Billing Wiki

▼ 👩‍🎤 Components (EM)

Note: diagram also highlights DAC blockchain elements as whole DAC is developed with close connection to DDC.

https://miro.com/app/board/uXjVLByDyVc=/?moveToWidget=3458764609468764389&cot=14

Components	Components execution	**Owner**	**Link to WIKI**
Client DDC SDK	⚒️ DDC Client SDK (JS) Execution @Ulad Palinski	@Ulad Palinski	🧰 DDC Client JS SDK Wiki @Ulad Palinski
DDC Core	💽 DDC Core Execution @Sergey Poluyan	@Ulad Palinski	🆕 DDC Core Wiki
DAC Core+Validation	🤖 DAC & Inspection Execution @Yahor Tsaryk	@Sergey Poluyan	🎃 DAC & Inspection Wiki @Yahor Tsaryk
Developer Console	🙂Developer Console UI Execution @Anton	@Anton Mazhuto	
Cluster Applications	🎧 Cluster Applications Execution @Anton Mazhuto	@Anton Mazhuto	🎶 Cluster Applications Wiki @Anton Mazhuto 🖥️ Node Provider UI Wiki @Anton Mazhuto 🦑 Blockchain indexer
DDC Utility	🛠️ DDC Utility Execution	@Ayush Mishra	⚙️ DDC Utility Mission

Referenced CEF Knowledge

Describe how 🔲 DDC @Sergey Poluyan & ⚙️ Orchestrator Wiki (A3) are related

⛰️ CEF @Orchestrator Wiki (A3)

👩‍🎤 Strategy (PM)

🐲 DDC Mission / Strategy 2024 @Anonymous

👩‍🎤 Product Specifications (PM)

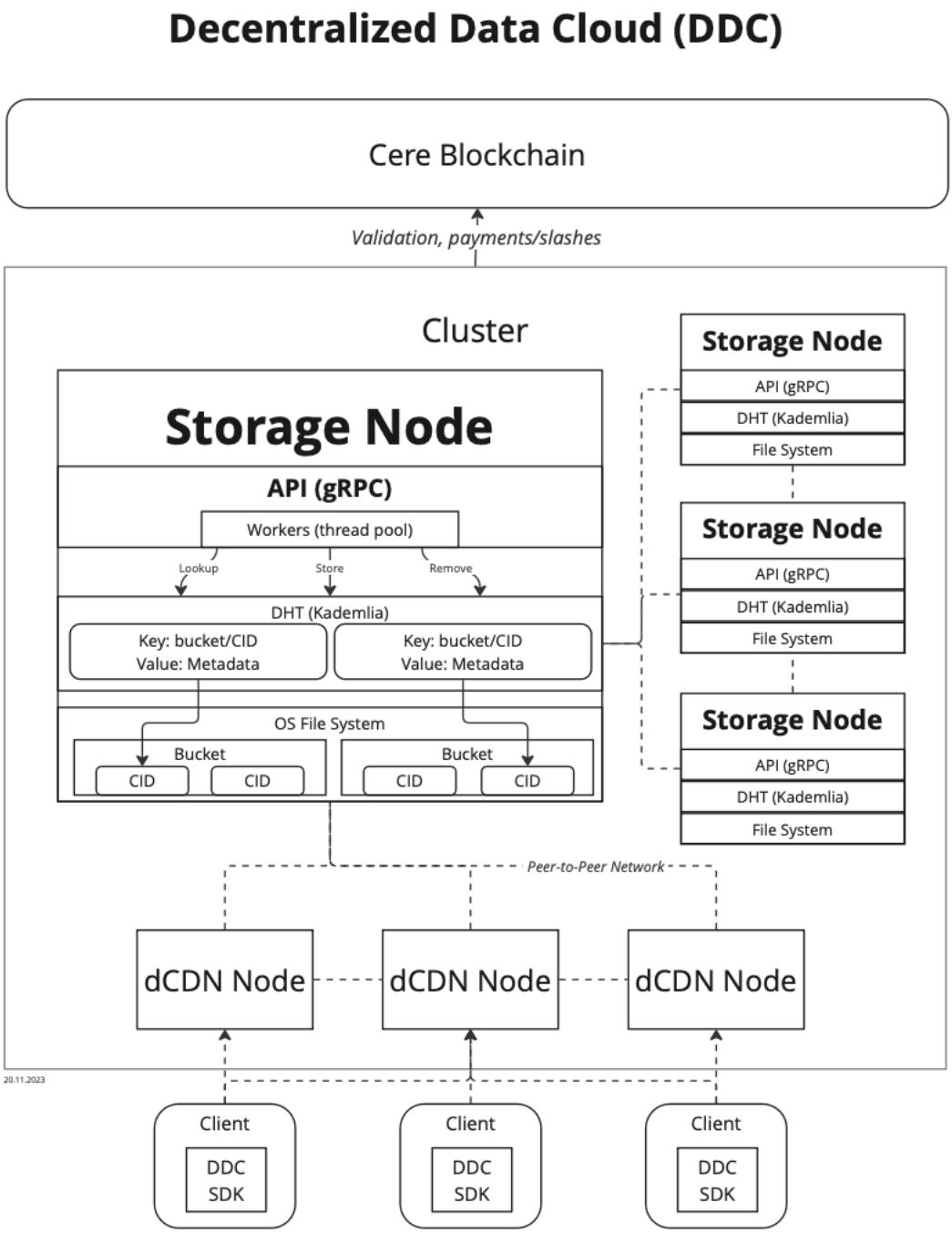
👩‍🎤 User Persona (PM)

DDC - User Personas

Aa Name	Interface	Objective	Role
🌐 Alex	CM UI	Wants to up and run a node to earn CERE tokens by running node in DDC	Node Provider
🌲 John	CM UI	Wants to run the cluster, manage nodes and earn CERE tokens	Cluster Manager
🤔 Max	DDC SDK Developer Console	Wants to	App developer

👩‍💻 User stories and cases (PM)

🏗️ Technical Specifications and System Architecture (EM)



- Architectural decision records

[ADR] Data redundancy strategy

[ADR] Efficient and secure large file access

[ADR] Support Merkle DAG's on API level

[ADR] CNS (Cere|Content Name System)

[ADR] MIME type support (Content-Type HTTP header)

[ADR] no-cache directive support (Cache-Control HTTP header)

[ADR] Protocol interoperability and future-proofing

[ADR] Authentication and authorization

[ADR] CDN routing state and efficient caching (caching layer)

[ADR] Local on-disk data migration between releases

[ADR] Data deletion

[ADR] P2P Authentication

[ADR] Public traffic rate limiting

[ADR] On-chain cluster configuration

[ADR] DAC decentralisation

[ADR] DAC aggregate verification

[ADR] Blockchain integration

[ADR] External nodes support

[ADR] Telemetry authentication

[ADR] DAC realtime feedback (WIP)

[ADR] Reputation system [incl. for routing]

[ADR] Data auditor

[ADR] DDC product dashboard

[ADR] HA Gateway

[ADR] S3-Compatible Gateway for DDC Storage

🧪 Tests and simulations

📊 Dashboards

- Infrastructure monitoring - https://dashboards.cere.io/d/c0edcbb2-cd3c-46cc-ab4e-24d2d542c07c/ddc-infrastructure-monitoring?orgId=1&from=now-6h&to=now&var-network=mainnet&var-cluster=0059f5ada35eee46802d80750d5ca4a490640511&refresh=5s
- Application monitoring - https://dashboards.cere.io/d/e00352dd-8f33-4f09-af4c-11e35ded1f56/ddc-application-monitoring?orgId=1&from=now-6h&to=now&var-network=devnet&var-cluster=7f82864e4f097e63d04cc279e4d8d2eb45a42ffa&var-node=storage-6
- Cache - https://dashboards.cere.io/d/dddcdhzexga2ob/ddc-cache?orgId=1&from=now-6h&to=now&var-network=devnet&var-cluster=7f82864e4f097e63d04cc279e4d8d2eb45a42ffa&var-node=storage-6&var-cache_type=$__all
- Logs - https://dashboards.cere.io/d/dd5dce2c-a862-4fce-ba1a-1b4e7773871c/ddc-logs?orgId=1&from=now-30m&to=now&var-network=devnet&var-cluster=7f82864e4f097e63d04cc279e4d8d2eb45a42ffa&var-node=$__all
- P2P Monitoring - https://dashboards.cere.io/d/bdcux7eiev4sga/ddc-p2p-monitoring?orgId=1&from=now-6h&to=now&var-network=devnet&var-cluster=7f82864e4f097e63d04cc279e4d8d2eb45a42ffa&var-server_id=$__all&var-client_id=$__all&var-status=4&var-method=$__all

- Dragon 1
  - Overview - https://dashboards.cere.io/d/mainnet-public/ddc-dragon-1?orgId=1&from=now-6h&to=now

📦 Repositories

DDC is structured as a single storage node repo (see here to get module details). The other repos here are utility tools, currently only deployed for Dragon 1 .

▼ **DDC Core @Ulad Palinski**

▼

```
https://github.com/Cerebellum-Network/ddc-storage-node
```

▼ **Modules within the repo:**

- Routing https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/routing
- DAC https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/activity
- Authentication https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/auth
- Blockchain indexer https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/blockchain
- Cache https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/cache
- CNS https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/cns
- DAG https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/dag

- Erasure coding https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/erasure
- File storage https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/file
- HTTP router https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/http
- Data migration https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/migration

▼ **Deployments:**

Inventory: https://dashboards.cere.io/goto/UjqBfeiNg?orgId=1

Deployment scripts: https://github.com/Cerebellum-Network/ansible-playbook-repos/tree/master/ddc

▼ https://github.com/Cerebellum-Network/ddc-storage-node/tree/dev/internal/activity

Part of Storage node repo.

Deployment:

- Deployed as part of storage node
- Blockchain nodes

▼ **UI Layer @Anton Mazhuto**

▼

- https://github.com/Cerebellum-Network/developer-console-client

  The Developer Console Client project is structured as Monorepo with main business logic living in the src directory with multiple supplementary packages in packages folder.

  - https://github.com/Cerebellum-Network/cere-ddc-sdk-js

    DDC SDK is the client dependency to communicate with DDC and Cere Blockchain. It is integrated to the client using DDC SDK

package (https://www.npmjs.com/package/@cere-ddc-sdk/ddc-client)

- https://github.com/cere-io/cere-wallet-client

  Cere Wallet is used for authentication and transaction signing. It is integrated to the client app using Cere Wallet SDK package (https://www.npmjs.com/package/@cere/embed-wallet)

- https://github.com/Cerebellum-Network/cere-squid-indexer

  Cere Blockchain indexer project which is used by the client to sync client state with blockchain information: `balance`, `ddc deposit` , `buckets`

- https://github.com/Cerebellum-Network/faucet-service

  Cere Blockchain faucet service is used to transfer tokens during onboarding

- https://github.com/cere-io/cluster-management-service

  Cluster Manager service is used as backend part of the client. Later we might split it to include only Developer Console related functionality

▼ **Deployment:**

▼ MVP deployment

- Dev
  - https://developer-console-dev.core-dev.aws.cere.io/
- Stage
  - https://developer-console-stg.core-stage.aws.cere.io/
- Prod
  - https://developer-console.core.aws.cere.io/

- Dev - Deployed
  - https://dev.developer.console.cere.network/
- Stage - Deployed

    - https://stage.developer.console.cere.network/
  - Prod - Deployed
    - https://developer.console.cere.network/

  Deployment Script:

▼

▼ Frontend apps

▼ **Modules within the repo:**

- Node Provider (https://github.com/Cerebellum-Network/cluster-apps/tree/dev/apps/node-provider)
- Developer console (https://github.com/Cerebellum-Network/cluster-apps/tree/dev/apps/developer-console)
- Global registry (https://github.com/Cerebellum-Network/cluster-apps/tree/dev/apps/global-registry)

▼ Dependencies

The Cluster Apps project is structured as Monorepo with main business logic living in the src directory with multiple supplementary packages in apps folder. The packages refer to following repos:

- https://github.com/Cerebellum-Network/cere-ddc-sdk-js

  DDC SDK is the client dependency to communicate with DDC and Cere Blockchain. It is integrated to the client using DDC SDK package (https://www.npmjs.com/package/@cere-ddc-sdk/ddc-client)

- https://github.com/cere-io/cere-wallet-client

  Cere Wallet is used for authentication and transaction signing. It is integrated to the client app using Cere Wallet SDK package (https://www.npmjs.com/package/@cere/embed-wallet)

- https://github.com/Cerebellum-Network/cere-squid-indexer

Cere Blockchain indexer project which is used by the client to sync client state with blockchain information: `balance`, `ddc deposit` , `buckets`

- https://github.com/Cerebellum-Network/faucet-service

  Cere Blockchain faucet service is used to transfer tokens during onboarding

- https://github.com/cere-io/cluster-management-service

  Cluster Manager service is used as backend part of the client. Later we might split it to include only Developer Console related functionality

```
https://github.com/Cerebellum-Network/developer-console-client
```

▼ Backend apps

There is a backend service which currently stays in its own repository.

```
https://github.com/cere-io/cluster-management-service
```

Deployment: Not deployed yet

▼ **Tools and Utilities**

▼

```
https://github.com/Cerebellum-Network/cere-squid-indexer
```

▼ **Deployment:**

Squid API https://github.com/Cerebellum-Network/kubernetes-blockchain/tree/develop/charts/squid-api

Squid processor https://github.com/Cerebellum-Network/kubernetes-blockchain/tree/develop/charts/squid-processor

Deployment Script:

▼

- https://github.com/Cerebellum-Network/cere-ddc-sdk-js/

  This is a mono-repository project that includes all the DDC SDK modules.

👷 Deployments

DDC is deployed as a single repo:

https://github.com/Cerebellum-Network/ddc-storage-node

Docker repository: https://hub.docker.com/r/cerebellumnetwork/ddc-storage-node/tags

Inventory: https://dashboards.cere.io/goto/UjqBfeiNg?orgId=1

Dashboards: https://dashboards.cere.io/dashboards/f/f422a7ba-ac12-42d9-b139-b2a2f5dc8c5f/ddc

Deployment scripts: https://github.com/Cerebellum-Network/ansible-playbook-repos/tree/master/ddc

▼ Dragon 1 Deployment

Releases: https://github.com/Cerebellum-Network/ddc-storage-node/releases

Nodes inventory: https://github.com/Cerebellum-Network/ansible-playbook-repos/blob/master/ddc/inventory/mainnet

Inventory: https://dashboards.cere.io/goto/UjqBfeiNg?orgId=1

Public dashboard: https://dashboards.cere.io/d/mainnet-public/ddc-dragon-1?from=now-6h&to=now

- https://github.com/Cerebellum-Network/developer-console-client

  The Developer Console Client project is structured as Monorepo with main business logic living in the src directory with multiple supplementary packages in packages folder.

- https://github.com/Cerebellum-Network/cere-ddc-sdk-js

  DDC SDK is the client dependency to communicate with DDC and Cere Blockchain. It is integrated to the client using DDC SDK package (https://www.npmjs.com/package/@cere-ddc-sdk/ddc-client)

- https://github.com/cere-io/cere-wallet-client

  Cere Wallet is used for authentication and transaction signing. It is integrated to the client app using Cere Wallet SDK package (https://www.npmjs.com/package/@cere/embed-wallet)

- https://github.com/Cerebellum-Network/cere-squid-indexer

  Cere Blockchain indexer project which is used by the client to sync client state with blockchain information: `balance`, `ddc deposit` , `buckets`

- https://github.com/Cerebellum-Network/faucet-service

  Cere Blockchain faucet service is used to transfer tokens during onboarding

- https://github.com/cere-io/cluster-management-service

  Cluster Manager service is used as backend part of the client. Later we might split it to include only Developer Console related functionality

🚀 Releases

🚀 DDC | Releases

https://github.com/Cerebellum-Network/ddc-api-e2e-simulations/actions

📺Infrastructure

💻 DDC | Infrastructure

? How to guide

🏎️ Get started with DDC

DDC Runbook @Sharan Konerira @Sergey Poluyan

📕 The DDC Storage & CDN Node Onboarding Guide (DRAFT)

🏃 DDC | Runbook

⬛ How to launch a DDC Node Guide

🗝️ Key Values

- How are we encrypting data on DDC?
- How are all data interactions attributed to the user data wallet
- Who pays for the data persisted on DDC attributed to the data wallet
  - if it's the app → then what happens if the app stops paying
  - if the data is lost → does the user then truly own the data?

💁‍♀️ Questions

▼ Open questions:

1. Question 1
2. Question 2 (if it is a block - sync it to the Blockers section)

▼ Closed questions:

1. Question 1
2. Question 2

👩‍🎤 Additional links and references

💎 DDC | Resilience

🧯 DDC | Post mortem reports

📦 DDC | Archive

⚛️ DDC - DSC Community stack

▼ Subpages

⚖️ Protocol

🛠️ Tools

🖥️ Node Provider UI Wiki @Anton Mazhuto

🦑 Blockchain indexer

🧰 DDC Client JS SDK Wiki @Ulad Palinski

🆕 DDC Core Wiki

Routing & Metadata Distribution @Ulad Palinski

🎃 DAC & Inspection Wiki @Yahor Tsaryk

DAC internal/external api specs @Anonymous

🎶 Cluster Applications Wiki @Anton Mazhuto

🧑‍🚀 Cluster Management Toolset @Alisher Khassanov (archive)

👷 DDC: Infra provider

DDC: PR/FAQs

Stability Testing Wiki

⚙️ DDC Utility Mission

Verifiable DAC Results Wiki @Krishna Singh

🧰 DDC Client JS SDK Wiki @Ulad Palinski

💰 DDC Pricing & Billing Wiki

🚍 Developer Console UI Wiki @Anton Mazhuto